

2303A52416

batch=35

assignment=16.1

### Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
  - o Insert a new student record.
  - o Fetch all courses enrolled by a student.
  - o Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student–course relationships.

```
1 import sqlite3
2
3 conn = sqlite3.connect("student.db")
4 cursor = conn.cursor()
5
6 cursor.execute("CREATE TABLE IF NOT EXISTS Students(student_id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")
7 cursor.execute("CREATE TABLE IF NOT EXISTS Courses(course_id INTEGER PRIMARY KEY, course_name TEXT)")
8 cursor.execute("CREATE TABLE IF NOT EXISTS Enrollments(enrollment_id INTEGER PRIMARY KEY, student_id INTEGER, course_id INTEGER)")
9
10 # Insert student
11 cursor.execute("INSERT INTO Students VALUES (1, 'Akshaya', 20)")
12
13 # Insert course
14 cursor.execute("INSERT INTO Courses VALUES (101, 'AI')")
15
16 # Enroll student
17 cursor.execute("INSERT INTO Enrollments VALUES (1, 1, 101)")
18
19 # Fetch courses for student
20 cursor.execute("""
21 SELECT course_name FROM Courses
22 WHERE Enrollments.student_id = Enrollments.enrollment_id
23 WHERE student_id = 1
24 """)
25
26 # Count students in each course
27 cursor.execute("SELECT course_id, COUNT(student_id) FROM Enrollments GROUP BY course_id")
28
29 # Commit and close
30 conn.commit()
31 conn.close()
```

## Aim

To design a **Student Information System database** using Python and SQLite to store student, course, and enrollment information.

## Explanation

1. The program uses the **sqlite3 module** to connect Python with an SQLite database.
2. A database named **student.db** is created.
3. Three tables are created:
  - **Students** → stores student details like ID, name, and age.

- **Courses** → stores course information.
  - **Enrollments** → connects students with courses.
4. The **CREATE TABLE** command creates tables if they do not already exist.
  5. The program inserts sample data using **INSERT INTO** statements.
  6. A **JOIN query** is used to retrieve courses enrolled by a specific student.
  7. A **GROUP BY query** counts the number of students in each course.
  8. Finally, `commit()` saves the changes and `close()` closes the database connection.

## Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
  - List all appointments for a specific doctor.
  - Retrieve patient history by patient ID.
  - Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

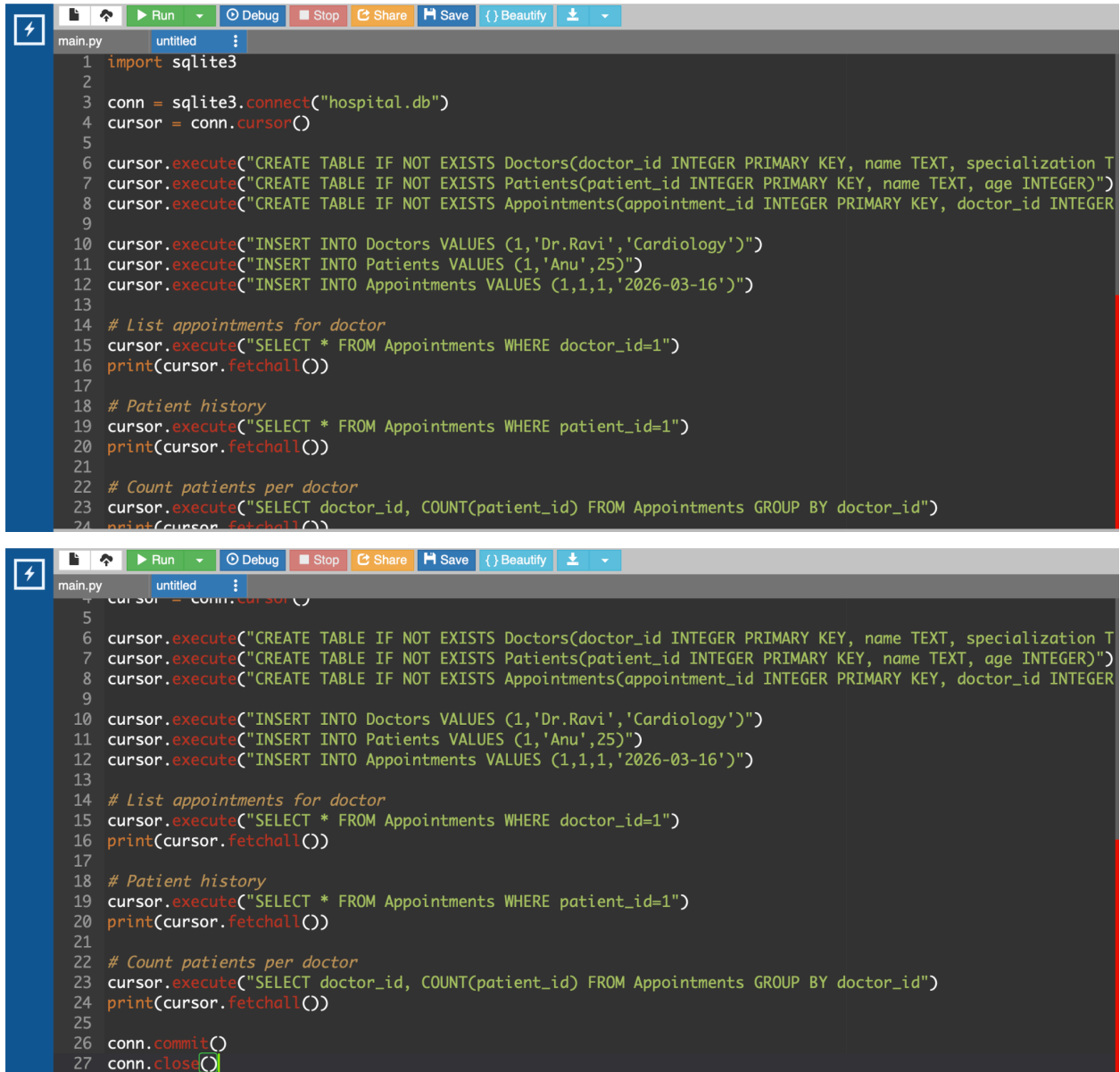
```

1 import sqlite3
2
3 conn = sqlite3.connect("shop.db")
4 cursor = conn.cursor()
5
6 cursor.execute("CREATE TABLE IF NOT EXISTS Users(user_id INTEGER PRIMARY KEY, name TEXT)")
7 cursor.execute("CREATE TABLE IF NOT EXISTS Products(product_id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER)")
8 cursor.execute("CREATE TABLE IF NOT EXISTS Orders(order_id INTEGER PRIMARY KEY, user_id INTEGER, product_id INTEGER, quantity INTEGER)")
9 cursor.execute("CREATE TABLE IF NOT EXISTS OrderDetails(order_detail_id INTEGER PRIMARY KEY, order_id INTEGER, product_id INTEGER, quantity INTEGER)")
10
11 cursor.execute("INSERT INTO Users VALUES (1, 'Akshaya')")
12 cursor.execute("INSERT INTO Products VALUES (1, 'Laptop', 50000)")
13 cursor.execute("INSERT INTO Orders VALUES (1, 1, '2026-03-16')")
14 cursor.execute("INSERT INTO OrderDetails VALUES (1, 1, 1, 2)")
15
16 Orders by user
17 cursor.execute("SELECT * FROM Orders WHERE user_id=1")
18 print(cursor.fetchall())
19
20 Most purchased product
21 cursor.execute("""
22 SELECT product_id, SUM(quantity)
23 FROM OrderDetails
24 GROUP BY product_id
  
```

## Explanation

1. The program creates a database called **library.db**.
2. Three tables are created:
  - **Books** → stores book ID, title, and author.
  - **Members** → stores member information.
  - **Loans** → records borrowed books and loan dates.

3. Data is inserted into all tables.
4. A **JOIN query** retrieves the titles of books currently issued.
5. The query connects the **Books** and **Loans** tables using the book ID.
6. Another query counts how many books each member has borrowed using **GROUP BY**.
7. The results are printed on the screen.



```
1 import sqlite3
2
3 conn = sqlite3.connect("hospital.db")
4 cursor = conn.cursor()
5
6 cursor.execute("CREATE TABLE IF NOT EXISTS Doctors(doctor_id INTEGER PRIMARY KEY, name TEXT, specialization TEXT)")
7 cursor.execute("CREATE TABLE IF NOT EXISTS Patients(patient_id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")
8 cursor.execute("CREATE TABLE IF NOT EXISTS Appointments(appointment_id INTEGER PRIMARY KEY, doctor_id INTEGER, patient_id INTEGER, appointment_date TEXT)")
9
10 cursor.execute("INSERT INTO Doctors VALUES (1,'Dr.Ravi','Cardiology')")
11 cursor.execute("INSERT INTO Patients VALUES (1,'Anu',25)")
12 cursor.execute("INSERT INTO Appointments VALUES (1,1,1,'2026-03-16')")
13
14 # List appointments for doctor
15 cursor.execute("SELECT * FROM Appointments WHERE doctor_id=1")
16 print(cursor.fetchall())
17
18 # Patient history
19 cursor.execute("SELECT * FROM Appointments WHERE patient_id=1")
20 print(cursor.fetchall())
21
22 # Count patients per doctor
23 cursor.execute("SELECT doctor_id, COUNT(patient_id) FROM Appointments GROUP BY doctor_id")
24 print(cursor.fetchall())
25
26 conn.commit()
27 conn.close()
```

## Explanation

1. The program connects to a database named **hospital.db**.
2. Three tables are created:

- **Doctors** → stores doctor ID, name, and specialization.
  - **Patients** → stores patient details.
  - **Appointments** → records appointment details including doctor, patient, and date.
3. Sample records are inserted into each table.
  4. SQL queries are executed using the **execute() function**.
  5. The query **SELECT \* FROM Appointments WHERE doctor\_id=1** retrieves all appointments for a specific doctor.
  6. Another query retrieves patient appointment history.
  7. A **GROUP BY query** counts the number of patients treated by each doctor.